

Programmazione 1 - Elena Giunta – A.A. 2023-2024 Canale F-N

Identificatori

Gli "**Identificatori**" sono nomi **univoci** assegnati nella dichiarazione di variabili, tipi, funzioni nel programma. Caratteristiche e limitazioni:

→ Non è possibile usare parole chiave, essendo riservate per uso speciale (non puoi chiamare una variabile "int").

→ Devono iniziare con una lettera o con un underscore; quindi, NON possono iniziare con numeri ecc.

→ Non possono esserci spazi nei nomi e bisogna prestare attenzione al Case sensitive

Classi di memoria

Ogni **identificatore** in un programma ha degli attributi:

- **Classe di memoria:** determina la sua **permanenza** in memoria. Gli **specificatori** di classe di memoria sono **auto, register, extern, static**.
- **Permanenza in memoria:** è il periodo durante il quale l'identificatore esiste nella memoria.

Permanenza in memoria automatica: Si usa la parola **auto**. Le variabili esistono finché il controllo del programma esce dal blocco. Un esempio sono le variabili locali, nelle quali si omette la parola auto.

Permanenza in memoria statica: Si usano le parole **extern** e **static**. Le variabili **static** esistono dall'inizio fino alla fine del programma, ma l'accesso ad esse non è sempre possibile. Le variabili **globali** e i nomi di funzione sono della classe di memoria **extern** per impostazione predefinita.

- **Campo di azione:** lo scope è dove l'identificatore può essere usato in un programma. Un campo d'azione può essere:
 - **Esteso alla funzione**
 - **Esteso al file**
 - **Esteso al blocco**
 - **Esteso al prototipo di funzione**
- **Collegamento:** il collegamento di un identificatore **determina**, per un programma con diversi file sorgente, **se l'identificatore è conosciuto soltanto nel file sorgente corrente o in qualunque file sorgente** con le opportune dichiarazioni.

Numerazione

Standard IEEE 754

Per rappresentare i numeri nei calcolatori si usa la rappresentazione in base 2. I **numeri reali** sono **rappresentati** nel formato in **virgola mobile** (floating point), cioè una rappresentazione compatta che deriva dalla rappresentazione scientifica. Lo **standard IEEE 754** è il più diffuso per la rappresentazione dei numeri in virgola mobile, avendo a disposizione word di **32 o 64 bit**:

1 bit per la rappresentazione del **segno**

8 o 11 bit per la rappresentazione dell'**esponente**

23 o 52 bit per la rappresentazione del significando o **mantissa**

Lo standard IEEE 754 riserva alcune **combinazioni di bit per** la rappresentazione di **valori speciali**.

- **±Inf: Divisione** di numeri **per zero**, oppure casi di **overflow** (positivo o negativo)
- **NaN:** Forme indeterminate (risultato indefinito)
- **±0 Underflow** codificati come zero con segno, a seconda che il risultato della espressione sia positivo o negativo.

Si verifica underflow quando il numero da rappresentare, in valore assoluto, è troppo piccolo per essere rappresentato. Positivo: Numero positivo troppo piccolo per essere rappresentato. Negativo: Numero negativo troppo piccolo per essere rappresentato

—>**Overflow:** il valore assoluto del numero è maggiore del numero più grande rappresentabile. Positivo: Risultato troppo grande. Negativo: Risultato positivo troppo piccolo

Vi sono **errori di approssimazione** quando il numero non è rappresentabile con numero finito di cifre o il numero di cifre significative del numero da rappresentare è maggiore della precisione p

La **precisione p** di una codifica floating point è rappresentata dal numero di cifre significative che è possibile rappresentare nella mantissa. • 6 cifre per la precisione singola —> 15 cifre per la precisione doppia. La **mantissa** (o significando) in virgola mobile è generalmente espressa in forma normalizzata. Si moltiplica o si divide la codifica binaria della mantissa per una certa potenza di 2. In tal modo la rappresentazione della mantissa rimarrà solo una cifra prima della virgola, cioè 1. Inoltre, dato che la cifra prima della virgola è sempre 1, questa non viene rappresentata, risparmiando così 1 bit.

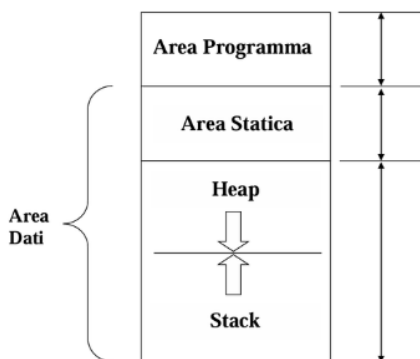
Il valore dell'esponente si rappresenta con il valore k detto bias: $E = e + k$. **E** è il valore effettivamente rappresentato nel campo esponente. —> **k** è il bias = +127 —> **e** è il valore esponente ottenuto durante il calcolo della mantissa.

La memoria

La memoria è divisa in **quattro sezioni**:

- Area del **codice**, contiene il codice del programma
- Area **stack**, contiene i record di attivazione delle funzioni (variabili locali e parametri)
- Area **heap** disponibile per allocazioni dinamiche
- Area **statica**, contiene le variabili statiche

L'Heap e lo Stack non hanno una dimensione fissa, la loro dimensione varia in base all'uso. L'area complessiva per entrambi è fissa e viene determinata durante l'esecuzione del programma.



Record di attivazione della memoria

La **pila delle chiamate delle funzioni**, (area stack) o **memoria automatica** è una struttura dati che gestisce le chiamate e i ritorni delle funzioni in un programma e le variabili locali. Funziona secondo il principio last-in first-out (LIFO).

La zona di memoria che viene creata per una singola funzione viene detta *record di attivazione*, e contiene i suoi parametri formali e le sue variabili locali. Quando una **funzione viene chiamata**, un nuovo "**record di attivazione**" viene inserito con un "push" nella pila. La zona di memoria per la funzione main si trova nel punto più basso dello stack, ogni volta che si chiama una funzione, viene creato un record immediatamente sopra. Questo **record contiene l'indirizzo di ritorno** per far tornare il controllo alla funzione chiamante una volta completata l'esecuzione.

I record di attivazione gestiscono le variabili locali (automatiche) delle funzioni. Quando la funzione chiamata torna alla funzione chiamante, il suo record di attivazione viene rimosso con un "pop" dalla pila e quindi cancellato. Le variabili locali non servono più, quindi la memoria può essere liberata e riusata per creare altri record di attivazione.

Se avvengono più chiamate di funzioni di quanti record di attivazione possono essere memorizzati nella pila delle chiamate, si verifica un errore irreversibile, ovvero **stack overflow**. In C, il concetto di stack e heap è strettamente legato all'allocazione di memoria, soprattutto nell'allocazione dinamica.

Heap

L'heap è un segmento di memoria adibito all'allocazione dinamica, ed è gestita esplicitamente dal programmatore, mediante funzioni come **malloc**, **calloc** e **realloc** definite in `stdlib.h`. Può essere considerato come una struttura dati albero astratto. È gestito dal sistema operativo.

L'allocazione dinamica è usata per riservare memoria contenere determinati valori, ed assegnarne l'indirizzo di partenza ad un puntatore. In questo modo è possibile richiedere l'allocazione di un'area di memoria la cui dimensione è nota solo in fase di esecuzione (non conosciamo la grandezza del dato, che quindi non può essere allocato staticamente).

La funzione `malloc()` restituisce un puntatore void all'area di memoria allocata, quindi può essere assegnato a ogni tipo di puntatore con un cast, convertendolo nel tipo desiderato. Nel caso in cui non si riesca ad allocare memoria, la funzione ritorna `NULL`. La funzione `calloc()` svolge lo stesso compito della `malloc()`, ma inizializza tutta la memoria allocata a zero. Per ottenere il cambiamento della dimensione di un'area precedentemente allocata è necessario ricorrere alla `realloc()`. Il contenuto del vecchio blocco viene copiato nel nuovo blocco e quindi non vengono perse informazioni.

La memoria allocata nell'heap non segue la struttura LIFO dello stack, persiste anche dopo che una funzione è terminata, fino a quando il programmatore dealloca esplicitamente la memoria allocata che non è più necessaria, utilizzando la funzione **`free()`**, o quando l'intero programma non termina. L'heap può causare problemi di segmentazione della memoria se non gestita correttamente.

Memoria Statica

L'allocazione statica accade quando dichiarare una variabile statica o globale. Ogni variabile statica o globale definisce un blocco di spazio, di dimensione fissa. Lo spazio viene allocato una volta, quando il programma viene avviato nel momento dell'esecuzione, e non viene mai liberato. La memoria viene deallocata automaticamente quando il programma termina.

I file

I file memorizzano a lungo termine i dati. Ogni file è uno stream sequenziale di byte, che termina con un marcatore di end-of-file o dopo uno specifico numero di byte. In C i file vengono distinti in due categorie: **file di testo** (costituiti da caratteri ed organizzati in linee (ciascuna terminata da `'\n'`)) e **file binari** (sequenze "grezze" di byte).

Nel Sistema Operativo è definita una **struttura dati FILE** che contiene:

- Il nome del file
- Posizione corrente su file (indicante il prossimo byte da leggere o scrivere su file);
- Modalità di utilizzo del file (lettura, scrittura o lettura e scrittura);
- Un indicatore di end-of-file, indicante il raggiungimento della fine del file

Il Sistema Operativo gestisce il **descrittore di file**, ovvero l'indice di un array del sistema operativo chiamata **tabella dei file aperti**. Ogni elemento dell'array contiene un **File Control Block (FCB)** per amministrare un file.

Per poter leggere o scrivere su un file è necessario dichiarare un puntatore a FILE. Quando un file viene aperto, a esso viene associato uno stream che fornisce il canale di comunicazione tra file e programma. Quando inizia l'esecuzione di un programma, il sistema operativo apre automaticamente anche i puntatori allo standard input, output ed error.

I byte di un file sono sequenzialmente raggruppati in **blocchi** detti **record**. Un record è un blocco di byte che costituisce un elemento (numero, stringa, struttura, etc.). Per accedere ad uno specifico record è necessario posizionarsi al byte da dove inizia. Una chiave del Record, ad esempio, identifica un record per facilitare il ritrovamento di uno specifico record da un file.

File ad Accesso Sequenziale:

- I record sono di **lunghezza variabile**
- **Accesso Sequenziale:** L'accesso ai dati avviene in modo sequenziale, senza la possibilità di accedere direttamente a posizioni specifiche nel file. Per accedere al record numero n bisogna leggere tutti i precedenti n-1 record per determinare il byte da dove esso inizia.
- **Operazioni di Base:** I dati vengono letti o scritti nel file utilizzando funzioni di input/output come fscanf() per la lettura e fprintf() per la scrittura. Se vengono modificati, i dati possono anche essere sovrascritti e quindi persi. Bisogna chiudere correttamente il file utilizzando fclose().

File ad Accesso Casuale:

- I record sono di **lunghezza fissa**.
- **Accesso Diretto:** È possibile accedere direttamente ed individualmente a qualsiasi record nel file senza dover leggere tutti i record precedenti.
- **Aggiornamento Efficienti:** I dati possono essere aggiornati o cancellati senza distruggere gli altri dati già inseriti.
- **Operazioni di Base:** Per leggere o scrivere dati in posizioni specifiche nel file, vengono utilizzate funzioni come fread() e fwrite() o fseek().

Puntatori e puntatori a puntatori

I puntatori sono delle variabili che non contengono un valore un semplice valore numero intero o alfanumerico, bensì un indirizzo della locazione di memoria dove è memorizzato un'altra variabile. Il tipo di una variabile di un puntatore è "puntatore a tipo". Il suo valore è un indirizzo. La definizione di un puntatore avviene allo stesso modo di una variabile, con l'aggiunta di un asterisco tra il tipo e il nome della variabile.

L'operatore di indirizzo & non restituisce la variabile, ma il suo indirizzo di memoria

L'operatore di dereferenziazione *, se applicato ad una variabile, fa riferimento all'oggetto puntato.

- “*” in una dichiarazione: serve per dichiarare una var. di tipo puntatore (es. `int pi;`)
- “*” in una espressione: è l'operatore di dereferenziazione (es. `a = *pi;`)
 - “*” “&” uno inverso dell'altro: `*&a` equivale ad `a`

Puntatori a puntatori

Essendo i puntatori variabili come tutte le altri, si può determinare il loro indirizzo. Un puntatore a puntatore è una variabile che contiene l'indirizzo di un altro puntatore. Si aggiungono tanti asterischi in base a quanti livelli di indirectione sono necessari. Per accedere al valore finale, bisogna dereferenziare il puntatore tante volte quanti puntatori sono stati usati. Il puntatore è una variabile con un tipo, per cui il suo indirizzo è un puntatore a un puntatore. Per memorizzare per esempio l'indirizzo di un puntatore a intero in una variabile, questa deve essere di tipo puntatore a puntatore a intero. Questo tipo si scrive `int **`.

Passaggio degli argomenti per valore e per riferimento

Vi sono due modi di passare argomenti a una funzione: per **valore** e per **riferimento**. Tutti gli argomenti in C sono passati per valore. Quando un argomento viene passato per valore, viene creata una copia dell'argomento. La funzione riceve questa copia e qualsiasi modifica fatta all'argomento all'interno della funzione non influisce sul valore originale passato dalla funzione chiamante.

Il passaggio per riferimento, in C può essere simulato tramite i puntatori. Quando si chiama una funzione con argomenti che devono essere modificati, vengono passati gli indirizzi degli argomenti con l'operatore di indirizzo (&) alla variabile (nella funzione chiamante). Nella funzione chiamata, poi useremo l'operatore “*” per accedere al contenuto della variabile.

Gli array non vengono passati usando l'operatore &, perché il C passa automaticamente la locazione iniziale nella memoria dell'array (il nome di un array è un puntatore costante al primo elemento dell'array).

Suddividere un programma in più files

Suddividere il codice in più file è utile per migliorare l'organizzazione, la manutenzione e il riutilizzo del codice nei progetti di grandi dimensioni. Ci sono tre metodi principali per farlo:

1. **Inclusione diretta di file sorgente e compilazione unica:** tutto il codice sorgente è incluso direttamente nel file principale e viene compilato in un'unica fase.
2. **Librerie statiche** collegate al programma durante la fase di compilazione e **librerie dinamiche** caricate in memoria solo quando necessario durante l'esecuzione del programma.
3. **Inclusione di header file e compilazione separata:** In questo metodo, le dichiarazioni delle funzioni e le definizioni delle strutture dati vengono messe in un file header (.h), mentre l'implementazione delle funzioni viene scritta in un file sorgente separato (.c). Per esempio, se si ha un file mylib.h, il cui contenuto è incluso nei file .c che definiscono le funzioni, bisogna utilizzare `#include "mylib.h"` per accedere alle dichiarazioni necessarie nel programma principale

La compilazione di un programma in C avviene in due fasi principali: la compilazione vera e propria e il linking.

- Il risultato della compilazione è un file oggetto, binario ma non ancora eseguibile.
- Il linkaggio è l'operazione che "aggancia" a quel file oggetto le librerie di sistema che lo rendono eseguibile sul PC.

La compilazione e il linking avvengono separatamente:

- Compilazione dei file sorgente .c in file oggetto.o con comandi come `gcc -c file.c`.
- Linking dei file oggetto per creare un eseguibile finale con `gcc -o programma file1.o`

Ricerca Sequenziale

La ricerca sequenziale, o lineare, è un algoritmo di ricerca per trovare un elemento in un array non ordinato. Questo metodo esamina ogni elemento della lista, fino a trovare l'elemento desiderato, o fino a giungere alla fine della lista. Si inizia con il primo elemento e si confronta l'elemento corrente con l'elemento che si sta cercando:

→Se l'elemento corrente non è quello ricercato, si passa al successivo

→Se l'elemento corrente è quello ricercato, si è trovato l'elemento e l'algoritmo termina.

```
int ric_sequenziale(int* arr, int n, int x) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == x) {
            return i; // Ritorna l'indice se l'elemento è trovato
        }
    }
    return -1; // Ritorna -1 se l'elemento non è trovato
}
```

Ricerca dicotomica

La ricerca dicotomica o binaria è un algoritmo per trovare un elemento in una lista **ordinata**. Consiste nel confrontare l'elemento centrale dell'array con la chiave di ricerca. Ad ogni iterazione, l'algoritmo prosegue la ricerca solo su metà degli elementi considerati al passo precedente. Quindi dividiamo ripetutamente l'intervallo di ricerca a metà fino a che l'intervallo è ridotto a zero.

→Se sono uguali, l'elemento è stato trovato

→Se la chiave di ricerca è minore di x, la ricerca prosegue iterativamente nella prima metà dell'array. Quindi l'intervallo si aggiorna da "low" a "medio + 1"

→Se la chiave di ricerca è maggiore di x, la ricerca prosegue iterativamente nella seconda metà dell'array. Quindi l'intervallo si aggiorna da "high" a "medio - 1"

```
int binarySearch(int* array, int x, int low, int high) {  
    while (low <= high) {  
        int mid = low + (high - low) / 2;  
  
        if (array[mid] == x) { //elemento trovato  
            return mid;  
        }  
  
        // Se l'elemento a metà è minore di x, significa che x si trova nella metà destra  
        if (array[mid] < x) {  
            low = mid + 1;  
        } else { //Altrimenti, x si trova nella metà sinistra  
            high = mid - 1;  
        }  
    }  
    return -1;  
}
```

Algoritmi di Sorting

Insertion Sort

Divisione dell'Array: L'array è diviso in due parti:

1. **Parte ordinata:** inizialmente contiene solo il **primo** elemento dell'array.
2. **Parte non ordinata:** contiene tutti gli altri elementi a partire dal secondo.

Procedura di Ordinamento

1. **Scorrimento degli elementi non ordinati:** L'algoritmo scorre gli elementi della parte non ordinata uno per uno, partendo dal secondo elemento dell'array (indice i = 1).
2. **Selezione della chiave:** Ad ogni iterazione, l'elemento corrente della parte non ordinata viene preso come "chiave"

3. Confronto e inserimento:

- La chiave viene confrontata con gli elementi della parte ordinata (a sinistra della chiave).
- Se la chiave è maggiore dell'elemento confrontato nella parte ordinata, rimane nella sua posizione.
- Se la chiave è minore, gli elementi nella parte ordinata che sono maggiori della chiave vengono spostati di una posizione verso destra per fare spazio alla chiave.

4. **Espansione della parte ordinata:** Dopo aver trovato la posizione corretta per la chiave, la chiave viene inserita in quella posizione, espandendo così la parte ordinata di un elemento.

Cicli Utilizzati

1. **Ciclo for esterno:** Parte da $i = 1$ fino a N . In ogni iterazione, l'elemento alla posizione i è considerato la chiave (key). Itera su ogni elemento dell'array, a partire dal secondo, perché il primo è considerato già ordinato. In questo for dichiariamo anche l'indice $j = i - 1$, usato per scorrere la parte ordinata e trovare la posizione corretta per inserire key
2. **Ciclo while interno:** Trova la posizione corretta per inserire l'elemento corrente nell'array già ordinato. Sposta gli elementi più grandi di key verso destra per creare uno spazio vuoto dove inserire l'elemento corrente. La **Condizione $j \geq 0$** : Assicura che non si esca dai limiti dell'array durante il confronto.
 - **Condizione $array[j] > key$** : Controlla se l'elemento corrente nella parte ordinata è maggiore della chiave. Se sì, l'elemento viene spostato di una posizione a destra.
 - **Decremento di j ($j -= 1$)**: Si sposta all'elemento successivo verso sinistra nella parte ordinata.

Inserimento della Chiave: Una volta trovato l'elemento nella parte ordinata che non è maggiore della chiave, la chiave viene inserita nella posizione corretta ($array[j + 1]$).

```
int i, key, j;
```

```
for (i = 1; i < n; i++) {
    key = arr[i];
    j = i - 1;
    /* Move elements of array that are greater than key,
    to one position ahead of their current position */
    while (j >= 0 && arr[j] > key)
    {
        arr[j + 1] = arr[j];
        j = j - 1;
    }
    arr[j + 1] = key; //insert key in the correct position
}
```

Selection Sort

Divisione dell'Array: L'array è diviso in due parti:

1. **Parte ordinata:** inizialmente vuota.
2. **Parte non ordinata:** contiene tutti gli elementi dell'array.

Procedura di Ordinamento

1. **Scorrimento della parte non ordinata:** L'algoritmo scorre l'intera parte non ordinata per trovare l'elemento più piccolo.
2. **Selezione del minimo:** Per ogni iterazione, selezioniamo l'elemento più piccolo dalla parte non ordinata.
3. **Aggiornamento del minimo:** Se troviamo un elemento più piccolo dell'attuale minimo durante la scansione, aggiorniamo l'indice del minimo.
4. **Scambio degli elementi:** Alla fine della scansione, scambiamo l'elemento minimo trovato con l'elemento all'inizio della parte non ordinata (posizione i) utilizzando una variabile temporanea, espandendo così la parte ordinata di un elemento.

Cicli Utilizzati

1. **Ciclo for esterno:** Itera con l'indice i su ogni elemento dell'array, considerandolo come il primo elemento della porzione non ancora ordinata. Ad ogni iterazione, i indica l'indice dell'elemento che deve essere posizionato correttamente. Questo for arriva fino ad N-1 perché l'ultimo elemento sarà già ordinato al termine del ciclo precedente.
2. **Ciclo for interno:** Trova l'indice dell'elemento minimo nella porzione non ancora ordinata dell'array. Inizia da i+1 e procede fino ad N. Confronta l'elemento corrente con il valore minimo trovato finora. Se l'elemento corrente è più piccolo, aggiorna l'indice del minimo con una variabile tmp.

```
double tmp = 0;
for(int i = 0; i < n-1; i++) {
    int min = i;
    for(int j = i+1; j < n; j++) {
        if(Y[j] < Y[min]) {
            min = j;
        }
    }
    tmp = Y[min];
    Y[min] = Y[i];
    Y[i] = tmp;
}
```

Bubble sort

Il Bubble Sort è un algoritmo di ordinamento che ricorsivamente scorre la lista, confronta i due elementi vicini e li scambia se si trovano nell'ordine sbagliato.

Procedura di Ordinamento

1. **Scorrimento della lista:** L'algoritmo scorre la lista ripetutamente.
2. **Confronto degli elementi adiacenti:** Ad ogni iterazione, confronta gli elementi adiacenti (elemento j e $j+1$).
3. **Scambio degli elementi:**
 - Se l'elemento in posizione j è maggiore dell'elemento successivo ($j+1$), si scambiano con una variabile temporanea.
4. **Loop del processo:** Il ciclo continua fino a quando la lista non è completamente ordinata.

Cicli utilizzati

- **Indice i – Ciclo esterno:** Utilizzato per ripetere il processo di ordinamento. Parte da $i = 0$ e va fino a $N-1$ (dove N è la dimensione dell'array). Fermarsi a $n-1$ è un'ottimizzazione: se facessimo n passate, nell'ultima passata confronteremmo l'ultimo elemento con sé stesso, un'operazione inutile
- **Indice j – Ciclo interno:** Utilizzato per scorrere la lista, confrontare gli elementi adiacenti e scambiarli se sono nella posizione errata. Parte da $j = 0$ e va fino a $N-i-1$ per non confrontare elementi già ordinati nelle iterazioni precedenti.

```
int temp;
for (int i = 0; i < n - 1; i++) {
    for (int j = 0; j < n - i - 1; j++) {
        if (array[j] > array[j + 1]) {

            temp = array[j];
            array[j] = array[j + 1];
            array[j + 1] = temp;

        }
    }
}
```

Strutture Dati

Liste singolarmente collegate

Una lista collegata è una organizzazione lineare di strutture autoreferenziali, chiamate nodi. A una lista collegata si accede mediante un puntatore al primo nodo della lista. Le liste collegate sono dinamiche, pertanto la lunghezza di una lista può aumentare o diminuire in fase di esecuzione, in base alle necessità.

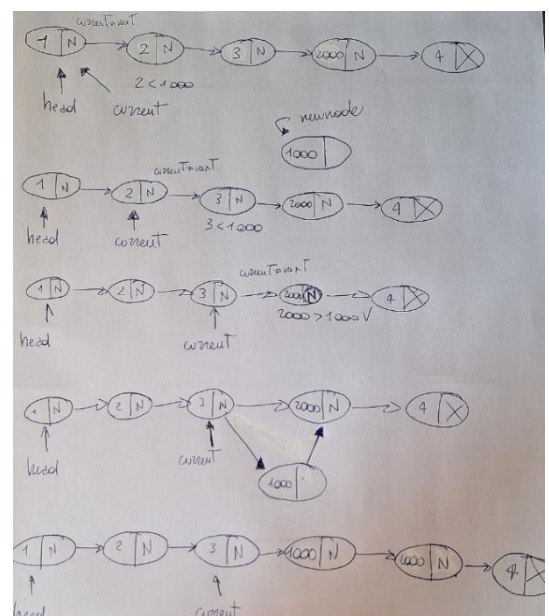
- Ogni nodo in una lista singolarmente collegata contiene un **valore** (o chiave) e un **puntatore al nodo successivo** (NEXT). Per convenzione, il puntatore di collegamento nell'ultimo nodo di una lista è posto a NULL, per segnare la fine della lista.
- È possibile scorrere la lista solo in **un senso**, cioè dalla testa alla coda.

Inserimento in lista: L'inserimento di un nuovo nodo in una lista avviene in diverse posizioni:

1. **Inizio della lista** (inserimento in testa).
2. **Primo nodo nella lista** (inserimento per lista vuota).
3. **In una posizione specifica** (inserimento ordinato o in una posizione specificata).

Per inserire un nodo in lista, il primo passo è assicurarsi che il nodo non sia il primo il lista, se è questo il caso, inizializziamo il puntatore next del newnode a NULL e spostiamo head al nuovo nodo, essendo l'unico in lista. Il secondo caso è l'inserimento in testa: il next di newnode punta al nodo che puntava la testa, e dopo questo passo spostiamo la testa al nuovo nodo. Se l'inserimento dovesse avvenire in una posizione specifica della lista, avremo bisogno di un puntatore d'appoggio, current, a cui assegneremo lo stesso valore di head, questo perché non possiamo assolutamente alterare la posizione di head. Dato che ci siamo accertati, tramite i controlli precedenti che il nodo da inserire non è sicuramente in testa, lavoriamo con il valore successivo a current, current->next.

Mediante un ciclo while, iteriamo nella lista il puntatore current, grazie al quale compariamo il dato in newnode ed i dati di current->next->data. Alla prima iterazione, ad esempio, compariamo 1000 con 2. Il newnode viene inserito in lista se il suo dato è più piccolo del dato presente in current->next->data. Essendo 1000 ovviamente più grande di 2, mandiamo avanti il puntatore current = current->next. Il processo si ripete fino a quando la condizione del while non si verifica più. Usciti dal ciclo while, per connettere il newnode alla lista, basterà far puntare la parte puntatore next del newnode al nodo successivo a current (quindi newnode->next = current->next) ed infine connettere la parte next di current al newnode (current->next = newnode)

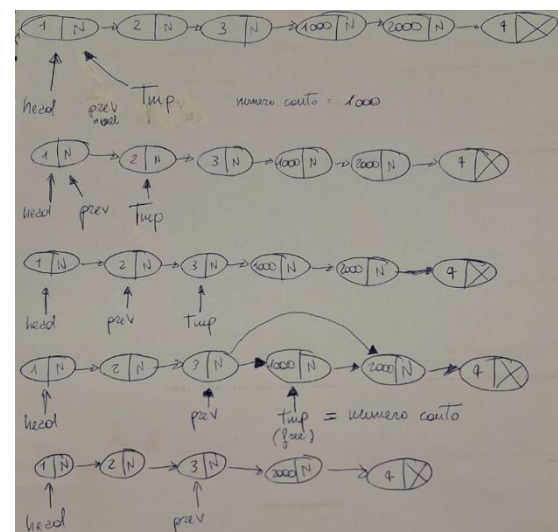


Codice per inserire nodi nella lista con ordinamento (funzione insert)

```
void insert(record r, lista* Lista) {  
  
    nodo* newnode = malloc(sizeof(nodo));  
    if(newnode == NULL) {  
        fprintf(stderr, "Memoria insufficiente");  
        exit(-1);  
    }  
  
    newnode->data = r;  
  
    //primo inserimento nella lista  
    if(Lista->head == NULL) {  
        newnode->next = NULL;  
        Lista->head = newnode;  
    } else { //inserimento in testa  
        if(newnode->data <= Lista->head->data) {  
            newnode->next = Lista->head;  
            Lista->head = newnode;  
        } else { //inserimento in lista  
            nodo* current = Lista->head;  
            while(current->next != NULL && newnode->data > current->next->data) {  
                current = current->next;  
            }  
            //trovata la posizione giusta, inserisci il nodo  
            newnode->next = current->next;  
            current->next = newnode;  
        }  
    }  
}
```

ELIMINAZIONE DI NODI DALLA LISTA

Per eliminare un nodo da una lista, solitamente passiamo un valore da ricercare nella parte "data" dei nodi: se il dato viene trovato, vuol dire che quel nodo è da cancellare. Dopo aver controllato che la lista non sia vuota, creo due variabili puntatore a nodo: tmp a cui assegno il valore di head, e prev a cui assegno NULL. Il primo caso è quello in cui il nodo da eliminare sia quello in testa: avendo copiato il valore della testa in tmp, spostiamo avanti il valore di head e facciamo una free di tmp. Se il nodo da cancellare si trova nella lista, iniziamo un ciclo while, comparando il valore puntato da tmp->data con quello che abbiamo passato alla funzione, per verificare che essi non siano uguali, conservando di volta in volta il valore di tmp in prev (prev = tmp) e mandando avanti tmp (tmp = tmp->next), finché non avremo trovato il valore corretto da eliminare. Se non abbiamo sfornato la lista (if tmp != NULL) colleghiamo il nodo a cui sta puntando attualmente prev, al nodo successivo a tmp. Quindi prev->next (il puntatore next di prev) = tmp->next (il nodo successivo a tmp). Ed infine free(tmp).



Codice per eliminare nodi in base ad un dato valore (funzione delete)

```
void delete(int numero_conto, lista* Lista) {

    if(Lista->head == NULL) {
        fprintf(stderr, "Lista vuota\\n");
        exit(-1);
    }

    //Puntatori temporanei per attraversare la lista
    nodo* tmp = Lista->head;
    nodo* prev = NULL;

    //nodo da eliminare è il primo nella lista
    if (tmp != NULL && tmp->data == data) {
        Lista->head = tmp->next;
        free(tmp);
        return;// come la pop
    }

    // Cerchiamo il nodo da eliminare nella lista
    while (tmp != NULL && tmp->data != data) {
        prev = tmp;
        tmp = tmp->next;
    }

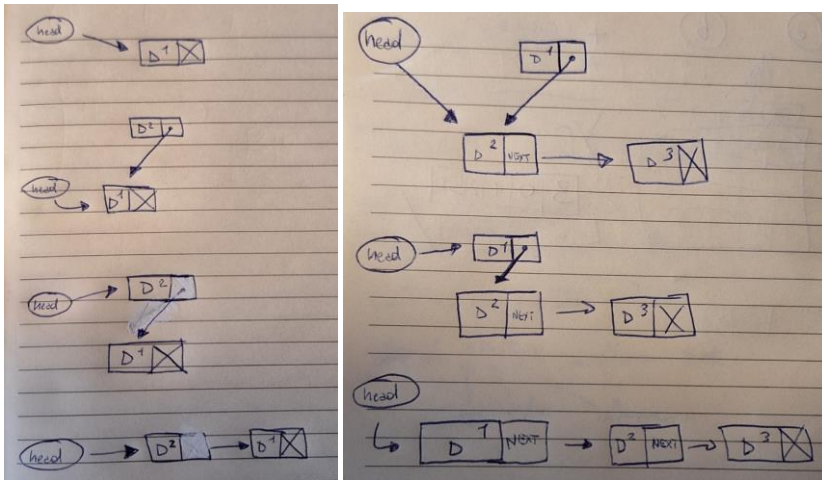
    //Se il nodo da eliminare è presente lo eliminiamo
    if (tmp != NULL) {
        //Aggiorniamo il puntatore del nodo
        //precedente al nodo da eliminare
        prev->next = tmp->next;
        free(tmp);
    }
}
```

Pila

Una pila è una struttura dati dinamica last-in, first out (LIFO) il primo elemento a uscire è l'ultimo entrato; quindi, è una versione vincolata di una lista collegata. A una pila si fa riferimento tramite un puntatore all'elemento in cima alla pila. È possibile aggiungere e rimuovere da una pila nuovi nodi solo in cima. Una pila ha principalmente due operazioni: Push: Aggiunge un elemento. Pop: Rimuove l'elemento. Il membro puntatore dell'ultimo nodo è posto a NULL per indicare il fondo della pila.

INSERIMENTO DI NODI IN TESTA (PUSH)

L'inserimento nella pila può avvenire in due modi: per pila vuota o per inserimento in testa. Se la pila è vuota, il valore "next" del newnode viene impostato a NULL, poiché appunto si tratta del primo nodo, mentre il valore head viene spostato in modo tale da puntare al nuovo nodo. Se si tratta invece di un inserimento in testa, quindi esistono già altri nodi nella pila, il puntatore next del nuovo nodo deve puntare al nodo che stava puntando head (newnode->next = Pila->head) e solo dopo questo passaggio possiamo spostare la testa, che punterà ovviamente a newnode (Pila->head = newnode).



Codice per inserire nodi in testa (funzione Push)

```
void push(Lista* Pila, float num) {

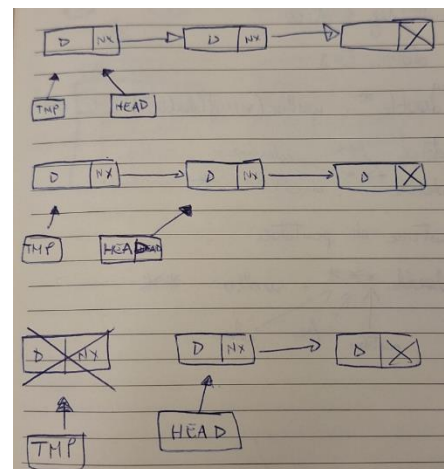
    nodo* newnode = (nodo*)malloc(sizeof(nodo));
    if (newnode == NULL) {
        fprintf(stderr, "Errore di allocazione della memoria\\n");
        exit(-1);
    }

    newnode->data = num;

    if(Pila->head == NULL) {
        newnode->next = NULL;
        list->head = newnode;
    } else {
        newnode->next = Pila->head;
        Pila->head = newnode;
    }
}
```

RIMUOVERE NODI DALLA TESTA (POP)

Dopo aver controllato che la pila non sia vuota, conservo in una variabile da ritornare il dato contenuto nel nodo puntato dalla testa (es. int ret = Pila->head->data). Dato che non possiamo cambiare il valore head, bisogna creare una variabile di appoggio tmp di tipo nodo puntatore, alla quale assegneremo lo stesso valore di head. Quindi, spostiamo head al nodo successivo (Lista->head->next o tmp->next): in questo modo, solo head si muoverà, mentre tmp punterà ancora alla "vecchia testa" (nodo in cima alla pila). Solo dopo aver spostato head, possiamo attuare una free di tmp, che libererà dalla memoria la vecchia testa. Ora la nuova testa sarà il nodo precedente a quello eliminato, a cui punta head poiché lo avevamo precedentemente spostato. Finite queste operazioni ritorniamo la variabile dato.



Codice per togliere nodi dalla testa (funzione Pop)

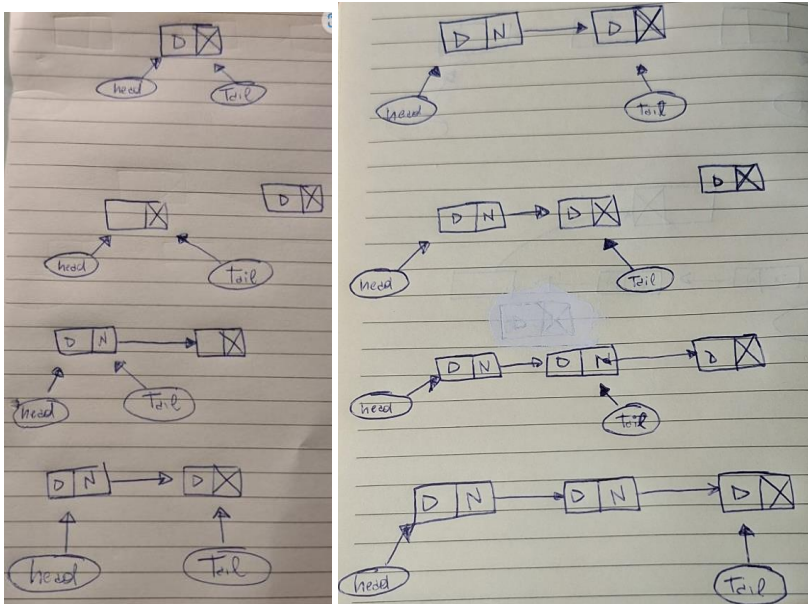
```
float pop(Lista* list) {  
    if(Pila->head == NULL) {  
        fprintf(stderr, "LISTA VUOTA\\n");  
        exit(-1);  
    }  
  
    float f = Pila->head->data;  
  
    nodo* tmp = Pila->head;  
    Pila->head = tmp->next;  
    free(tmp);  
    return f;  
}
```

Coda

La coda (queue) è una struttura dati dinamica utilizzata per gestire una raccolta di elementi in cui le operazioni di inserimento (enqueue) avvengono in una estremità (coda) e le operazioni di rimozione (dequeue) avvengono nell'estremità opposta (testa). La coda segue il principio FIFO (First In, First Out), cioè il primo elemento inserito è il primo a essere rimosso. Anche la coda pertanto è una versione vincolata della lista concatenata.

INSERIMENTO IN CODA

Dopo aver creato il nodo ed aver inserito in esso il dato, impostiamo subito il suo valore next puntatore a NULL. Per l'inserimento abbiamo, come per la pila, due casi: inserimento per coda vuota e inserimento in coda. Se la coda è vuota, avremo che il puntatore next di newnode sarà già impostato a NULL, quindi non ci rimane altro che far puntare sia Head che Tail a newnode. Se si tratta invece di un inserimento in coda, il valore next del nodo puntato da tail deve spostarsi al nuovo nodo, ed infine anche tail si sposta al nuovo nodo.



Codice per inserire nodi nella coda (funzione enqueue)

```
void enqueue (int num, Lista* list) {

    nodo* newnode = malloc(sizeof(nodo));
    if(newnode == NULL) {
        fprintf(stderr, "Memoria insufficiente per allocare il nodo\\n\\n");
        exit(-1);
    }

    newnode->data = num;
    newnode->next= NULL;

    if(list->tail == NULL){
        list->head = newnode;
        list->tail = newnode;
    }
    else{
        //il valore next puntato da tail deve puntare a newnode
        list->tail->next = newnode;
        list->tail = newnode; //sposta anche tail a newnode
    }

}
```

Codice per estrarre nodi dalla coda (dequeue = pop)

```
int dequeue (Lista* list) {
//la dequeue è uguale alla pop della stack, dato che toglie nodi dalla testa

    if(list->head == NULL) {
        fprintf(stderr, "Lista vuota\\n");
        exit(-1);
    }

    int numero = list->head->data;

    nodo* tmp = list->head; //copia testa in un tmp
    list->head = list->head->next; //sposta avanti la testa

    if(Coda->head == NULL){
        Coda->tail = NULL;
    } //aggiungi questa parte specialmente se il codice dà problemi

    free(tmp); //elimina il nodo che stava in testa

    return numero;
}
```

Matrici

Allocazione della Matrice di variabili

```
double** M = malloc(sizeof(double*) * a.n); //righe
for(int i = 0; i<a.n; i++) { //colonne
    M[i] = malloc(sizeof(double) * a.m);
}

int** A = malloc(sizeof(int*) * numero righe);
for(int i = 0; i < numero righe; i++) {
    A[i] = malloc(sizeof(int) * numero colonne);
}
```

Allocazione di Matrici di puntatori

```
int***M = malloc(sizeof(int**) * ag.n); //righe
for(int i = 0; i < ag.n; i++) {
    M[i] = malloc(sizeof(int*) * ag.m); //colonne
    for(int j = 0; j < ag.m; j++) {
        M[i][j] = malloc(sizeof(int));
    }
}

int*** A = malloc(sizeof(int**) * numero righe);
for(int i = 0; i < numero righe; i++) {
    A[i] = malloc(sizeof(int*) * numero colonne);
    for(int j = 0; j < numero colonne; j++) {
        A[i][j] = malloc(sizeof(int));
    }
}
```

Riempire la matrice

```
for(int i = 0; i<ag.n; i++) {
    for(int j = 0; j<ag.m; j++) {
        *(M[i][j]) = get_random()%(h+1); //viene aggiunto +1
        //perché l'operatore modulo % genera un numero compreso tra 0 e h
    }
}
```

Tipi in C

Data Type	Format Specifiers	Size
Signed char	%c	1Byte
Unsigned char	%c	1Byte
Int or Long int or Signed Long Int	%d	4 Bytes
Unsigned int Or unsigned long int	%u	4 Bytes
Short int	%hd	2 Bytes
Unsigned short int	%hu	2 Bytes
float	%f	4 Bytes
double	%lf	8 Bytes
Long Double	%Lf	10 Bytes

FUNZIONE FIBONACCI

```
int genFib(int n){
    if(n == 0 || n == 1){
        return n;
    }else{
        return genFib(n-1) + genFib(n-2);
    }
}
```

FUNZIONE FATTORIALE

```
int getFactorial(int x){
    if(x <= 0) {
        return 1;
    } else {
        return x * getFactorial(x-1);
    }
}
```